pandas.ipynb

Pandas - это мощная библиотека для анализа данных в Python, которая предоставляет удобные структуры данных и функции для работы с табличными данными. Она построена на основе NumPy и широко используется в области анализа данных, науки о данных и машинного обучения.

Установка Чтобы установить Pandas, используйте pip:
`pip install pandas`

Импорт библиотеки:
```
import pandas as pd
```

```
import pandas as pd
```

```

```

Основные структуры данных

Pandas предлагает две основные структуры данных:

Series: одномерный массив, который может содержать данные любого типа (целые числа, строки, числа с плавающей запятой и т. д.). Каждое значение в Series имеет индекс.

```
# Создание Series
l = [1, 2, 3, 4, 5]
s = pd.Series(l)
print(s)
```

```

```

DataFrame: двумерная таблица, состоящая из строк и столбцов. Каждый столбец может содержать данные разных типов. DataFrame можно рассматривать как контейнер для нескольких Series.

```
# Создание DataFrame
data = {
    'Имя': [['Аня', 'Катя'], 'Борис', 'Вика'],
    'Возраст': [25, 30, 22],
    'Группа': [1, 2, 3]
}

df = pd.DataFrame(data)
df
```

```
dict1 = {
    'ВКONE': ['Ася', 'Александра','Анна', 'Айдана'],
    'ВКIMB': ['Полина', 'Эдуард', 'Егор', 'Екатерина']
}
pd.DataFrame(dict1)
```

Основные операции с данными

Чтение и запись данных

Pandas поддерживает множество форматов для чтения и записи данных, включая CSV, Excel, SQL и другие.

```
# Пример
# Чтение данных из CSV файла
df = pd.read_csv('data.csv')

# Запись данных в CSV файл
df.to_csv('output.csv', index=False)
```

```
df = pd.read_csv('Данные.csv')
```

```
df
```

Чтение

CSV (Comma-Separated Values)

```
# Чтение CSV-файла
df = pd.read_csv('data.csv')

# Указать кодировку, если есть проблемы
df = pd.read_csv('data.csv', encoding='utf-8')

# Указать разделитель (если не запятая)
df = pd.read_csv('data.txt', sep='\t')  # для TSV
```

Excel

```
# Чтение Excel-файла
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Можно указать несколько листов или индекс
df = pd.read_excel('data.xlsx', sheet_name=0)  # первый лист
```

JSON

```
# Чтение JSON-файла
df = pd.read_json('data.json')

# Или из строки JSON
import json
json_str = '{"name": ["Alice", "Bob"], "age": [25, 30]}'
df = pd.read_json(json_str)
```

```

```

Запись

CSV

```
# Запись в CSV
df.to_csv('output.csv', index=False)  # index=False — без колонки индекса
```

Excel

```
# Запись в Excel
df.to_excel('output.xlsx', sheet_name='Data', index=False)
```

Основные методы

- head(): показывает первые n строк DataFrame.
- tail(): показывает последние n строк DataFrame.
- info(): выводит информацию о DataFrame, включая типы данных и количество ненулевых значений.
- describe(): предоставляет статистику по числовым столбцам

```

```

Индексация и фильтрация данных

Индексация

- **loc** для доступа по меткам.
- **iloc** для доступа по позициям.

```
# Индексация по меткам
print(df.loc[0])

# Индексация по позициям
print(df.iloc[0])
```

```
df['Количество детей'].argmax()
```

```

```

```
df[
    (df['Рост'] >= 170)&(df['Город'] == 'Москва')&(df['Профессия'] == 'Менеджер')&
(df['Возраст'] <= 30)&(df['Возраст'] >= 20)
    ]
```

```
df[df['Профессия'] == 'Менеджер']
```

```
df[['Рост', 'Вес', 'Количество детей']].corr()
```

Фильтрация

Вы можете фильтровать данные с помощью логических условий.

```
# Фильтрация по условию
young_people = df[df['Возраст'] < 30]
print(young_people)
```

Обработка отсутствующих данных

Pandas предоставляет функции для обработки отсутствующих данных.

- **isnull()**: проверяет, есть ли пропущенные значения.
- **dropna()**: удаляет строки с пропущенными значениями.
- **fillna()**: заполняет пропущенные значения.

```
# Проверка на пропущенные значения
print(df.isnull().sum())

# Удаление строк с пропущенными значениями
df_cleaned = df.dropna()

# Заполнение пропущенных значений
df_filled = df.fillna(0)
```

```
from matplotlib import pyplot as plt
```

```

```

Визуализация данных

Pandas интегрируется с библиотеками визуализации, такими как Matplotlib и Seaborn.

```
from matplotlib import pyplot as plt
# Построение графика
df['Возраст'].hist()
plt.title('Распределение возраста')
plt.xlabel('Возраст')
plt.ylabel('Частота')
plt.show()
```

Группировка данных:

```
data = {
    'Имя': ['Аня', 'Аня', 'Борис', 'Вика'],
    'Возраст': [34, 25, 30, 22],
    'Группа': [1, 4, 2, 3]
}

df = pd.DataFrame(data)
df
```

```
grouped = df.groupby('Имя').mean()  # Средние значения по Именам
grouped
```

```
df
```

```
df.groupby('Город')[['Зарплата','Вес','Рост']].mean()
```

```
df['Зарплата + премия'] = df['Зарплата'] + 0.5*df['Зарплата']
df
```

```
df.drop(columns=['Зарплата + премия'], inplace=True)
df
```

Добавление и удаление столбцов:

```
# Добавления
df['Рост'] = [170, 160, 165, 190]
df
```

```
# Удаление столбца
df.drop('Рост', axis=1, inplace=True)
df
```

Слияние и объединение:

```
df1 = pd.DataFrame({'key': ['A', 'B', 'C'], 'value1': [1, 2, 3]})
df1
```

```
df2 = pd.DataFrame({'key': ['A', 'B', 'D'], 'value2': [4, 5, 6]})
df2
```

```
df1 = pd.merge(df1,df2, on='key',how='outer')
```

```
merged = pd.merge(df1, df2, on='key', how='outer')  # Объединение по ключу
merged
```

```
df1.fillna(df1[['value1', 'value2']].mean())
```

```
import numpy as np
lamda = 5
np.random.poisson(lamda)
```

```

```

```
means = []
count_means = 1000

for i in range(count_means):
  x = []
  for j in range(1000):
    x.append(np.random.poisson(lamda))
  means.append(sum(x)/len(x))

plt.hist(means)
```

```
plt.hist(np.random.poisson(lamda,size=100000000).reshape(10000, 10000).mean(axis=1) , bins=50);
```

```
plt.hist(means)
plt.hist(x, alpha=0.2)
```

```
import matplotlib.pyplot as plt
import numpy as np

import matplotlib.animation as animation

fig, ax = plt.subplots()
#ax.set_xlim([-10, 10])

scat = ax.scatter(1, 0)
x = np.linspace(-10, 10, 10)


def animate(i):
    scat.set_offsets((x[i], np.sin(x[i])))
    return (scat,)


ani = animation.FuncAnimation(fig, animate, repeat=True, frames=len(x) - 1, interval=50)

# To save the animation using Pillow as a gif
writer = animation.PillowWriter(fps=15,
                                metadata=dict(artist='Me'),
                                bitrate=1800)
ani.save('scatter.gif', writer=writer)

plt.show()
```

```
df
```

$$f(x) = x^5 + 2x^4 + x + 1 \to min$$

```
x = np.linspace(-10,10)
y = x**5 + 2*x**4 + x + 1
x[np.argmin(y)]
plt.plot(x, y)
plt.scatter(x[np.argmin(y)],np.min(y))
```

```
# y = x^5 + 2x^4 + x + 1
# y' = 5x^4 + 8x^3 + 1
# y' =0
# 5x^4 + 8x^3 + 1 = 0
#
```